

Министерство цифрового развития, связи и массовых коммуникаций РФ
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра вычислительных систем
Допустить к защите
Зав. кафедрой к.т.н., доцент
_____ Перышкова Е.Н.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Разработка игры в шахматы с использованием
искусственного интеллекта

Пояснительная записка

Студент Стаценко Д.Е.

Институт ИВТ Группа ИБ-023

Руководитель Гонцова А.В.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	8
1 Постановка задачи	9
2 Теоретические сведения	10
2.1 Основные понятия теории игр	10
2.2 Типы игр.....	11
2.2.1 Кооперативные и некооперативные игры	11
2.2.2 Симметричные и асимметричные игры	11
2.2.3 Игры с полной и неполной информацией	11
2.3 Алгоритм принятия решения	12
2.4 Игры с нулевой суммой.....	12
2.5 Теорема минимакса.....	12
2.6 Алгоритм Minmax	13
2.7 Альфа/Бета отсечение	15
2.8 Алгоритм Negamax.....	15
3 Средства разработки	17
3.1 Язык программирования Java	17
3.2 Система автоматизации сборки приложения Gradle	18
3.3 Фреймворк JUnit.....	18
4 Программная реализация	20
4.1 Архитектура движка	20
4.2 Реализация алгоритма minmax.....	22
4.3 Реализация алгоритма Negamax	24
4.4 Эвристическая функция.....	26
5 Результат	28
ЗАКЛЮЧЕНИЕ	31
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	33
ПРИЛОЖЕНИЕ А	34
ПРИЛОЖЕНИЕ Б.....	36
ПРИЛОЖЕНИЕ В	39

ВВЕДЕНИЕ

Искусственный интеллект (ИИ) в шахматах является одной из самых актуальных и перспективных областей исследований. В последние время ИИ совершил значительный прогресс, что существенно повлияло как на весь мир, так и на шахматы в целом. Современные шахматные движки, такие как Stockfish и AlphaZero, могут играть наравне с чемпионами. Это позволяет изменить подход к обучению игры в шахматы как для новичков, так и для опытных игроков.

Шахматы, одна из настольных игр, на протяжении веков служат примером для демонстрации стратегического мышления и интеллектуального мастерства. С развитием технологий шахматы стали не только игрой, но и одним из предметов для исследований в области искусственного интеллекта. Разработка алгоритма для ИИ, способного играть в шахматы на высоком уровне, представляет собой уникальную возможность, предлагая новые подходы к решению сложных задач.

Так как в последнее время искусственный интеллект все больше привлекает к себе внимание, как со стороны обычных людей, так и технических специалистов, а исследование в этой области проводят по всему миру, то считаю, что данная тема наиболее актуальна в настоящее время.

1 Постановка задачи

Целью выпускной квалификационной работы является разработка игры в шахматы с использованием искусственного интеллекта и провести сравнительный анализ эффективности искусственного интеллекта на основе вычисленного рейтинга на сайте Lichess.org играя против других ботов сообщества.

2 Теоретические сведения

Теория игр как научная дисциплина изучает отношения между людьми, которые руководствуются несовпадающими (а иногда и противоположными) мотивами. Наряду с традиционными играми, такими как покер, шахматы, футбол и многие другие, теория игр изучает и такие серьезные отношения как рыночная конкуренция, загрязнение окружающей среды и т.д. В теории игр все эти серьезные отношения называют играми, поскольку в них, как и в играх, результат зависит от решений (стратегий) всех участников.

2.1 Основные понятия теории игр

Теория игр включает в себя несколько фундаментальных понятий, которые определяют структуру и динамику взаимодействий между участниками игры.

Игроки — это участники игры, принимающие решения с целью максимизации собственной выгоды или минимизации потерь. В рамках теории игр каждый игрок стремится выбрать наилучшую стратегию, учитывая возможные действия других участников.

Стратегии — это наборы возможных действий, которые могут быть выбраны игроком в каждой ситуации, возникающей в ходе игры. Стратегия формирует полный план действий игрока, определяя, какое решение он примет в каждой возможной игровой ситуации.

Выплаты — это результаты, которые получает каждый игрок в зависимости от выбранных стратегий. Выплаты могут быть выражены в различных формах, таких как денежные суммы, очки или иные единицы измерения, характеризующие выгоду или потери игрока.

Игра с нулевой суммой — это тип игры, в которой сумма выигрышей всех игроков в каждом возможном исходе равна нулю. В таких играх выигрыш одного игрока соответствует эквивалентным потерям другого игрока.

2.2 Типы игр

Игры могут различаться по нескольким критериям, которые определяют их структуру и правила взаимодействия между игроками.

2.2.1 Кооперативные и некооперативные игры

Кооперативные игры характеризуются возможностью формирования коалиций между игроками. В таких играх участники могут заключать соглашения о совместных действиях и распределении выигрышей, что позволяет им достигать коллективных целей.

Некооперативные игры предполагают, что каждый игрок действует автономно, без возможности заключения обязательных соглашений с другими участниками. В этих играх каждый игрок стремится максимизировать свою выгоду независимо от действий других игроков.

2.2.2 Симметричные и асимметричные игры

Симметричные игры характеризуются одинаковыми условиями для всех участников. В таких играх каждый игрок имеет равные возможности и правила, что создает равнозначные стартовые позиции для всех. Примером симметричной игры является игра "Камень, ножницы, бумага".

Асимметричные игры предполагают различия в ролях, возможностях и стратегиях игроков. В таких играх участники имеют неравные стартовые позиции и могут обладать различными ресурсами и целями, что влияет на их стратегические решения.

2.2.3 Игры с полной и неполной информацией

Игры с полной информацией предполагают, что все участники имеют полное знание о правилах игры, возможных стратегиях и выплатах для каждого из них. В таких играх отсутствует скрытая информация, и игроки принимают решения на основе полного представления о текущем состоянии игры.

Игры с неполной информацией характеризуются тем, что игроки не обладают полной информацией о стратегиях и выплатах других участников. В таких играх существует неопределенность, и игроки вынуждены принимать

решения, основываясь на ограниченных данных и вероятностных оценках возможных действий других игроков.

2.3 Алгоритм принятия решения

Алгоритмы принятия решений являются ключевой областью исследования в теории игр, экономике, управлении и искусственном интеллекте. Эти алгоритмы помогают моделировать и анализировать процесс выбора оптимальных действий в условиях неопределенности и множества возможных исходов. Основная цель таких алгоритмов — определить, какие действия приведут к наилучшим результатам с учетом различных факторов и ограничений.

Деревья решений: Визуальный и аналитический инструмент, который представляет возможные решения и их последствия в виде дерева. Ветки дерева показывают различные варианты действий и возможные исходы.

2.4 Игры с нулевой суммой

Игры с нулевой суммой — это особый класс игр в теории игр, где выигрыш одного игрока равен проигрышу другого. В таких играх сумма выигрышей и потерь всех игроков равна нулю, что означает, что нет возможности для обоих игроков одновременно улучшить свои позиции. Эти игры моделируют ситуации строгой конкуренции, где один игрок выигрывает за счет другого.

Шахматы как раз являются одним из примеров игры с нулевой суммой, где один игрок выигрывает, а другой проигрывает. Ничья также возможна, но суммарный выигрыш всегда остается нулевым.

2.5 Теорема минимакса

Теорема минимакса, предложенная Джоном фон Нейманом, является фундаментальной для игр с нулевой суммой. Она утверждает, что в любой игре с нулевой суммой с конечным числом стратегий для каждого игрока существует оптимальная стратегия, которая минимизирует максимальные потери игрока.

Формулировка: для двух игроков существует равновесие, где стратегия одного игрока минимизирует его максимальные потери, а стратегия другого максимизирует его минимальные выигрыши. Это равновесие называется равновесием в смешанных стратегиях.

2.6 Алгоритм Minmax

Minmax это алгоритм принятия решений в теории игр. Он используется для нахождения оптимальной стратегии в играх с нулевой суммой, где участвуют два игрока и один выигрывает, если другой проигрывает. Алгоритм строит дерево решений. Алгоритм рекурсивно проходит через дерево игры до достижения конечного состояния, оценивая каждое состояние игры и выбирая ход, который минимизирует потери при предположении, что противник выбирает свой лучший ход.

Алгоритм работает на построение дерева решения, глубина это дерева зависит от вычислительных мощностей. На рисунке 2.1 приведен пример работы алгоритма Minmax.

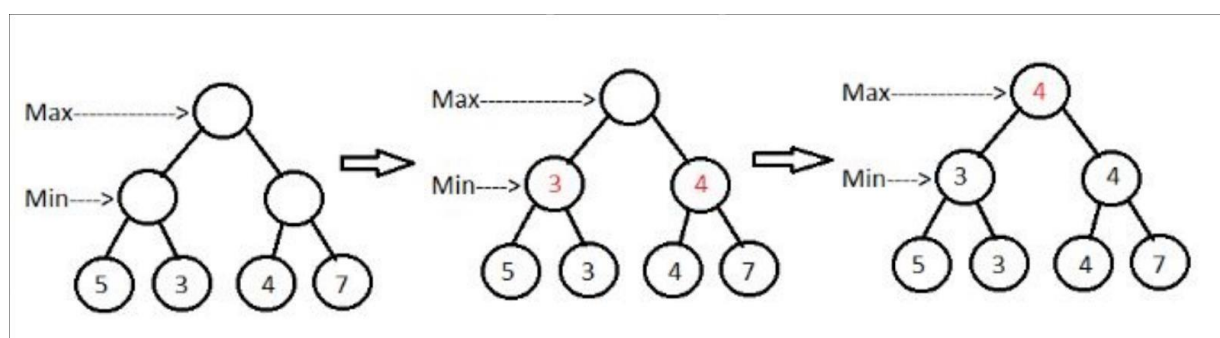


Рисунок 2.1 – Пример дерева minmax алгоритма

Каждый узел в дереве представляет собой конкретное состояние игры или положение фигур на шахматной доске после определенного хода. Ветви, исходящие из узла, обозначают возможные ходы, которые могут быть сделаны из данного состояния. Каждая ветвь ведет к новому узлу, представляющему новое состояние игры после выполнения хода. Корневой узел соответствует текущему состоянию игры, с которого начинается анализ. Листовые узлы представляют собой конечные точки дерева, где игра завершается (например, мат, пат или ничья) или где анализ прекращается на определенной глубине.

Оценка позиций обычно осуществляется с помощью эвристической функции. Эта функция присваивает числовое значение каждой позиции, основываясь на различных факторах, таких как материальное преимущество, позиционные соображения и другие стратегические элементы. В данном примере числа в узлах представляют собой оценку этих позиций.

Алгоритм Minmax чередует уровни Min и Max (листинг 2.1), так как игроки делают ходы по очереди. На уровне Max алгоритм стремится максимизировать выгоду ИИ. В контексте шахмат это означает выбор хода, который приводит к наиболее выгодной позиции для игрока. Алгоритм оценивает все возможные ходы и выбирает тот, который имеет наивысшую оценку с точки зрения стратегии и целей. На уровне Min алгоритм стремится минимизировать выигрыш оппонента. Это означает, что оппонент будет выбирать ходы, которые либо уменьшают потенциальный выигрыш ИИ, либо создают для него наибольшие затруднения.

Листинг 2.1 – Псевдокод алгоритма Minmax

```
function minimax(node, depth, maximizingPlayer) is
  if depth = 0 or node is a terminal node then
    return the heuristic value of node
  if maximizingPlayer then
    value := -∞
    for each child of node do
      value := max(value, minimax(child, depth - 1, FALSE))
    return value
  else (* minimizing player *)
    value := +∞
    for each child of node do
      value := min(value, minimax(child, depth - 1, TRUE))
    return value
```

2.7 Альфа/Бета отсечение

Альфа/Бета отсечение – это оптимизация алгоритма. Реализовано это так, что оценивание ветви дерева поиска может быть досрочно прекращено, если было найдено, что для этой ветви значение, оценивающей функцию в любом случае хуже, чем вычисление предыдущий ветви. На рисунке 2.2 приведен пример работы оптимизации альфа/бета отсечения.

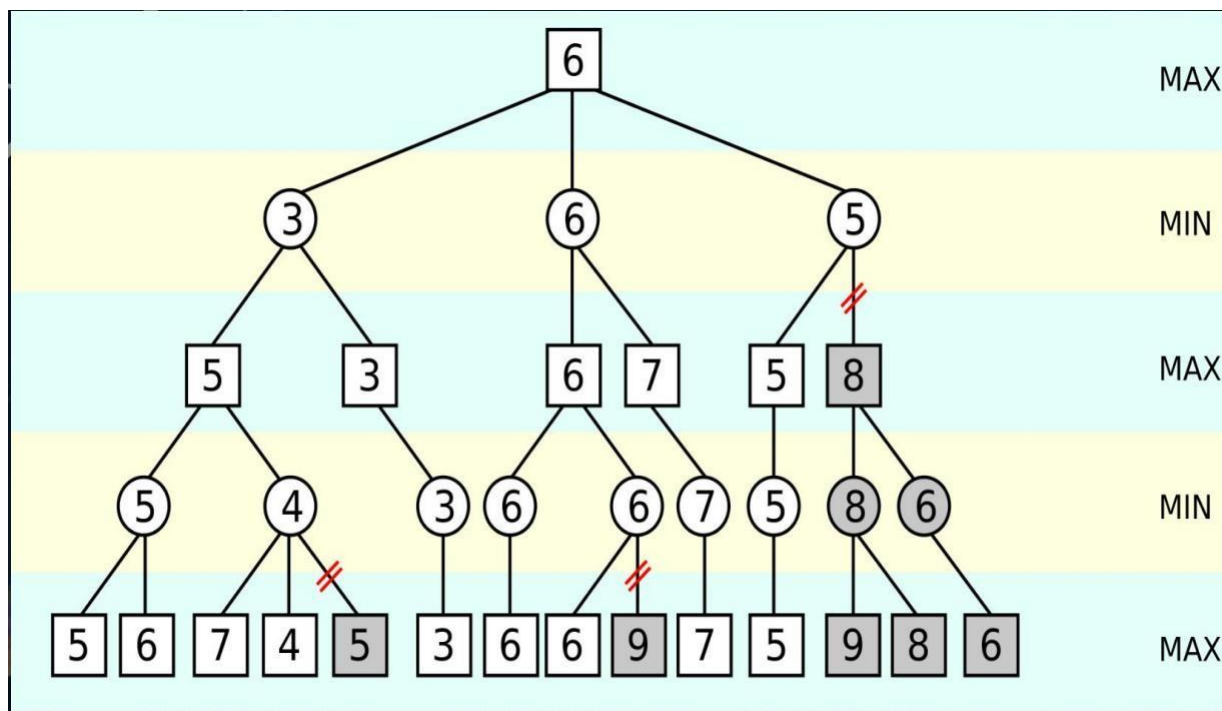


Рисунок 2.2 – Применение альфа/бета отсечения

2.8 Алгоритм Negamax

Алгоритм Negamax является вариантом (надстройкой) алгоритма Minmax, отличающимся более простой реализацией и улучшенной оптимизацией. В основе его идеи лежит концепция аналогичная алгоритму Minmax, но с упрощением за счет отсутствия разделения уровней на min и max. Вместо этого на каждом уровне всегда осуществляется поиск максимума.

Основное отличие алгоритма Negamax заключается в том, что на всех уровнях происходит поиск максимального значения, при этом эвристическая функция оценки должна возвращать значения с точки зрения текущего игрока узла. Например, в шахматной игре, если ход ИИ, эвристическая функция

должна возвращать положительное значение. Если ходит противник, функция должна возвращать отрицательное значение. Таким образом, чередование минимизации и максимизации заменяется на изменение знака числовых оценок.

Алгоритм Negamax начинается с инициализации текущего состояния доски и задания максимальной глубины для поиска. Затем генерируются все возможные ходы для текущего игрока. Алгоритм рекурсивно спускается по дереву ходов: если достигнута максимальная глубина или конечное состояние игры (мат, пат), то вычисляется эвристическая оценка текущего состояния доски. В противном случае для каждого возможного хода выполняется ход, и рекурсивно вызывается алгоритм Negamax для нового состояния доски с изменением знака возвращаемого значения (листинг 2.2).

Листинг 2.2 – Псевдокод алгоритма Negamax

```
function negamax(node, depth, color) is
  if depth = 0 or node is a terminal node then
    return color × the heuristic value of node
  value :=  $-\infty$ 
  for each child of node do
    value := max(value, -negamax(child, depth - 1, -color))
  return value
```

После оценки всех возможных ходов на каждом уровне выбирается максимальное значение среди всех оценок дочерних узлов, после изменения знака. Затем происходит подъем по дереву, передавая наилучшую оценку текущего уровня вверх. В корневом узле (начальное состояние доски) выбирается ход с наилучшей оценкой, и этот ход выполняется. На рисунке 2.3 приведен пример работы алгоритма Negamax.

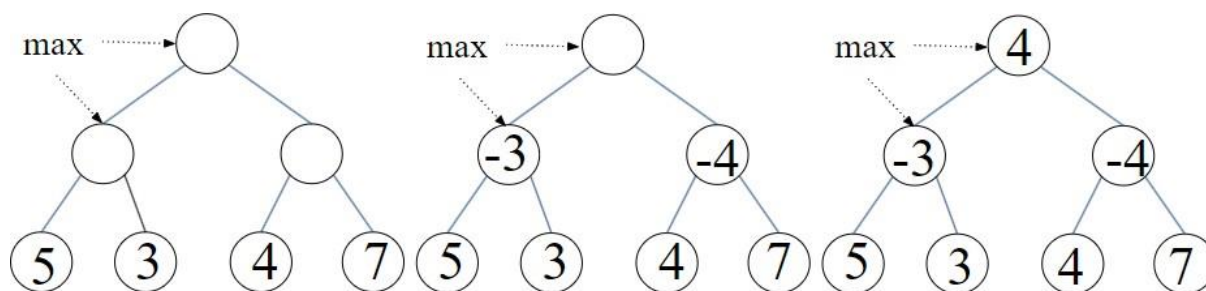


Рисунок 2.3 – Пример работы алгоритма Negamax

3 Средства разработки

3.1 Язык программирования Java

Язык Java высокоуровневый, объектно-ориентированный язык программирования, который был разработан компанией Sun Microsystems (ныне Oracle) и выпущен в 1995 году. Он обладает рядом характеристик, которые делают его популярным и удобным для различных видов разработки.

Основные особенности языка Java включают портативность, объектно-ориентированность, безопасность, многопоточность, богатую стандартную библиотеку и автоматическое управление памятью. Портативность Java обусловлена тем, что программы компилируются в байт-код, который исполняется виртуальной машиной Java (JVM), что позволяет запускать программы на любых устройствах с установленной JVM, независимо от операционной системы. Объектно-ориентированное программирование поддерживает моделирование сложных систем с использованием классов и объектов. Многопоточность позволяет создавать высокопроизводительные приложения, способные выполнять несколько задач одновременно. Java также предоставляет обширную стандартную библиотеку, которая включает множество классов.

Java подходит для разработки шахматного движка и ИИ по нескольким причинам. Во-первых, его портативность и кроссплатформенность позволяют запускать шахматный движок на различных платформах без необходимости изменять исходный код, что делает его доступным для широкого круга пользователей. Во-вторых, многопоточность Java позволяет эффективно распределять вычислительные ресурсы и улучшать производительность при анализе множества возможных ходов в шахматных играх.

Таким образом, Java является мощным и универсальным языком программирования, который идеально подходит для разработки сложных систем, таких как шахматные движки и искусственный интеллект.

3.2 Система автоматизации сборки приложения Gradle

Gradle - система автоматической сборки, построенная на принципах Apache Ant и Apache Maven, но предоставляющая DSL на языках Groovy и Kotlin вместо традиционной XML-образной формы представления конфигурации проекта.

В отличие от Apache Maven, основанного на концепции жизненного цикла проекта, и Apache Ant, в котором порядок выполнения задач (targets) определяется отношениями зависимости (depends-on), Gradle использует направленный ациклический граф для определения порядка выполнения задач.

Gradle был разработан для расширяемых многопроектных сборок, и поддерживает каскадную модель разработки, определяя, какие компоненты дерева сборки не изменились и какие задачи, зависящие от этих частей, не требуют перезапуска. Имеет свой собственный Gradle Daemon - фоновый процесс для ускорения сборки проекта.

Gradle был выбран в качестве системы автоматизации сборки приложения из-за его гибкости, мощных возможностей управления зависимостями и интеграции с средой разработки.

3.3 Фреймворк JUnit.

JUnit это популярный фреймворк для модульного тестирования на языке Java. Он позволяет писать и выполнять автоматические тесты для проверки корректности работы отдельных частей программного обеспечения. JUnit играет важную роль в процессе разработки, способствуя созданию надежного и качественного кода.

JUnit упрощают написание тестов, позволяя разработчикам определять методы, которые будут выполняться до и после тестов, а также сами тестовые методы. Тестовые случаи представляют собой отдельные методы, проверяющие конкретное поведение кода. Тестовые наборы объединяют несколько тестовых случаев для выполнения вместе.

Таким образом, JUnit является мощным и универсальным инструментом для модульного тестирования на Java, который помогает создавать надежное и качественное программное обеспечение. Его простота, поддержка автоматизации, интеграция с инструментами сборки и IDE, а также возможности тестирования различных аспектов кода делают JUnit отличным выбором для разработки и тестирования Java-приложений.

4 Программная реализация

4.1 Архитектура движка

Архитектура шахматного движка, в виде диаграммы классов (рисунок 4.1).

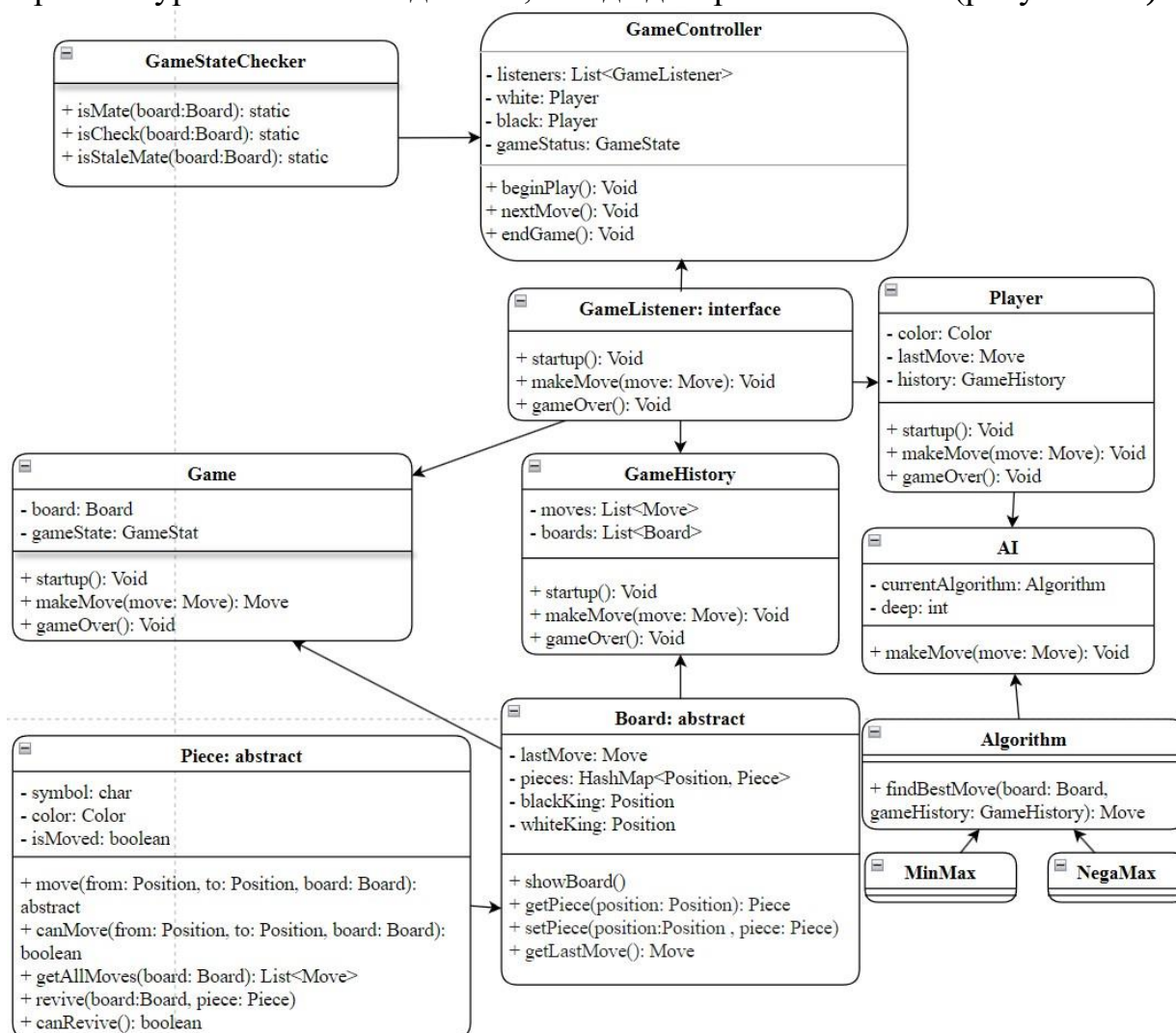


Рисунок 4.1 – Диаграмма классов движка шахмат.

Архитектура организована таким образом, чтобы обеспечить модульность, удобство тестирования и расширяемость. Она включает несколько основных компонентов:

Контроллер игры: центральный элемент, управляющий ходом игры, взаимодействующий с игроками и отслеживающий состояние игры. Этот компонент инициирует начало игры, управляет ходами и завершением игры. В контроллере есть, так называемые слушатели, это сама игра, история игры и оба игрока. Каждое событие, а именно, начало игры, каждый ход и конец игры,

запускает определенный метод в каждом из слушателей. Например игра, то есть класс Game, делает ход на общей доске, меняет состояние доки и текущий статус, допустим с хода белых на ход черных. История игры, GameHistory, сохраняет каждый ход и каждое состояние доски в массив, это нужно для ИИ, чтобы тот мог делать ходы наперед, а после быстро возвращается к старому состоянию. Игроки же, класс Player, по очереди делают ход.

Игроки и ИИ: представляют игроков, как человеческих, так и искусственных (ИИ). Игроки взаимодействуют с доской через контроллер игры, делая ходы и реагируя на изменения состояния игры. ИИ использует алгоритм для принятия решений и выполнения ходов.

Доска и фигуры: абстракции, представляющие шахматную доску и шахматные фигуры. Эти компоненты управляют состоянием доски, положением фигур и возможными ходами. Доска храниться в виде хэш таблицы, где ключом является позиция фигуры, по x и y, а значение сама фигура, и общий процесс происходит так, получаем фигуру и вызываем у объекта фигуры метод move, если по такой позиции фигуры нет, запрашиваем ход еще раз.

История игры: компонент, который сохраняет последовательность ходов и состояний доски на протяжении всей игры. Это позволяет осуществлять откат ходов, анализировать игру и сохранять ее историю.

Проверка состояния игры: компонент, который проверяет текущие состояния игры, такие как шах, мат и пат. Он используется для определения завершения игры и проверки допустимости ходов. Происходит такая проверка, после каждого хода.

Алгоритмы ИИ: реализуют логику принятия решений для ИИ. Эти алгоритмы анализируют текущую позицию на доске и выбирают наилучший ход в зависимости от заданных критериев.

Взаимодействие между компонентами происходит следующим образом: контроллер игры управляет ходом игры, оповещая игроков и слушателей о начале игры, сделанных ходах и завершении игры. Игроки, в

свою очередь, взаимодействуют с доской, делая ходы. Доска обновляет свое состояние и оповещает контроллер игры о выполненных ходах. История игры сохраняет все сделанные ходы и состояния доски для возможного анализа и отката. Проверка состояния игры обеспечивает корректность хода, проверяя на наличие шаха, мата и пата. Алгоритмы ИИ анализируют текущую позицию и предлагают наилучшие ходы.

4.2 Реализация алгоритма minmax

Алгоритм состоит из шагов:

1. Получить текущее состояние доски.
2. Выполнить все возможные ходы для текущего игрока, чередуя флаги max и min в зависимости от хода игрока, до достижения заданной глубины (например, 3).
3. Для каждого хода, после достижения максимальной глубины или конечного состояния игры, вычислить эвристическую оценку текущего состояния доски.
4. Если текущий узел обозначен как max, выбрать максимальное значение среди всех оценок дочерних узлов. Если текущий узел обозначен как min, выбрать минимальное значение среди всех оценок дочерних узлов.
5. Возвращаться на один уровень вверх по дереву, повторяя шаг 4 до тех пор, пока не будет обработано все дерево.
6. В корне дерева выбрать ход с наилучшей оценкой и выполнить его.

Использование альфа-бета отсечения позволяет избежать большого количества лишних вычислений. Благодаря этому, время выполнения одного хода не превышает 5 секунд, это видно на рисунке 4.2.

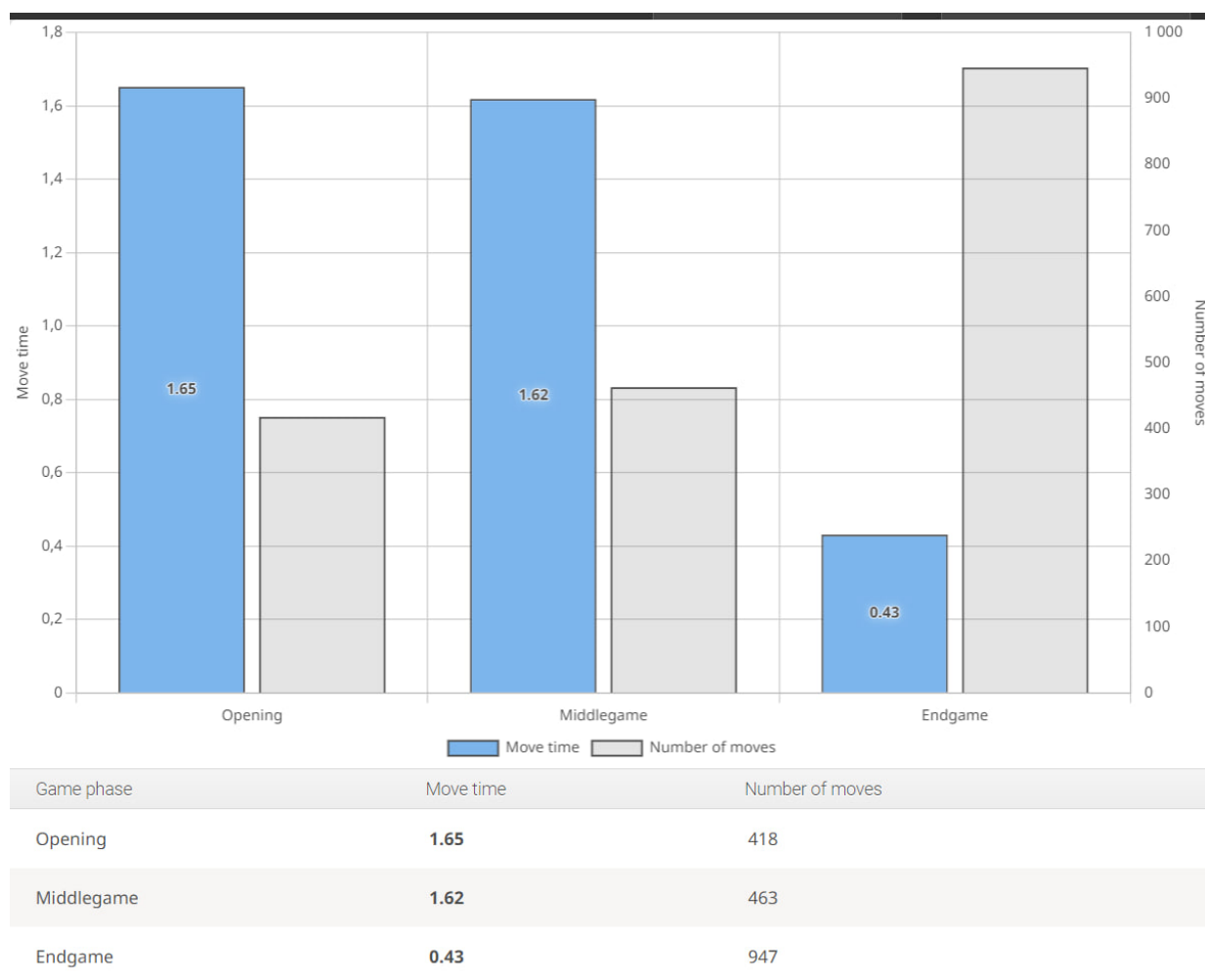


Рисунок 4.2 – Среднее время хода ИИ в разные стадии игр.

Этот график был взят с сайта Lichess.org, где тестировался разработанный ИИ. На нем видно, что среднее время хода в начале и середине игры одинаковое и примерно равно 1.63, что является хорошим показателем, так как именно в эти стадии игр происходит больше всего вычислений, за счет большого количества действующих фигур на доске. А вот в конце, когда в матче осталось не так много фигур, время хода меньше секунды, а если быть точнее полсекунды.

В реализации алгоритма Minmax с оптимизацией методом альфа-бета отсечения максимальная достигнутая глубина равна 3 (листинг 4.1). При глубине большей трех ход будет выполняться значительное количество времени из-за большого объема вычислений.

Листинг 4.1 – Код алгоритма minmax

```
private Node minmax(Board board, GameHistory gameHistory,
Color color,
    int deep, double alpha, double beta, boolean isMax) {
    if (deep == 0 || isGameOver(board, gameHistory)) {
        double eval = Evaluation.evaluationFunc(board,
gameHistory, botColor, isMax);
        return new Node(board.getLastMove(), eval);
    }

    List<Move> allMoves = getPossibleMoves(board, color);
    Node bestMove = new Node(allMoves.get(0), isMax ? MIN_EVAL
: MAX_EVAL);
    if (deep == this.deep) {
        this.bestMove = bestMove;
    }

    for (Move move : allMoves) {
        Board copyBoard = copyAndMove(move, board);
        GameHistory copyHistory = copyHistoryAndMove(copyBoard,
gameHistory);
        double eval =
            minmax(copyBoard, copyHistory, color.getOpposite(),
deep - 1, alpha, beta, !isMax).eval;

        if (isMax) {
            if (eval > alpha) {
                alpha = eval;
                bestMove.move = move;
                bestMove.eval = eval;
            }
        } else {
            if (eval < beta) {
                beta = eval;
                bestMove = new Node(move, eval);
            }
        }

        if (beta <= alpha) {
            break;
        }
    }

    return bestMove;
}
```

4.3 Реализация алгоритма Negamax

Суть алгоритма такая же, как и у minmax, только за одним условием. Вместо использования флагов max и min алгоритм чередует знаки результата

вычисления эвристической функции. Это позволяет сократить объем кода и упрощает процесс распараллеливания.

Для параллельной обработки задач алгоритм использует ForkJoinPool из стандартной библиотеки Java. ForkJoinPool использует принцип «разделяй и властвуй», то есть сначала делим всё дерево на подзадачи, выполняем их, потом объединяем результаты и делаем это всё рекурсивно для каждого поддерева.

Каждый уровень дерева поиска разделяется на подзадачи, которые выполняются в отдельных потоках. После завершения выполнения всех подзадач и объединения их результатов в один массив, выбирается максимальное значение из полученных чисел.

Реализация алгоритма Negamax (листинг 4.2) включает в себя два основных этапа: рекурсивный поиск по дереву ходов и параллельное выполнение подзадач.

Листинг 4.2 – Код алгоритма Negamax

```
private Node negamax(
    Board board, GameHistory gameHistory, Color color, int
    deep, double alpha, double beta) {
    if (deep == 0 || isGameOver(board, gameHistory)) {
        double signEval = color == botColor ? 1.0 : -1.0;
        boolean isBotMove = color == botColor;

        return new Node(
            board.getLastMove(),
            Evaluation.evaluationFunc(board, gameHistory,
            botColor, isBotMove) * signEval);
    }

    List<Move> moves = getPossibleMoves(board, color);
    Node bestMove = new Node(moves.get(0), MIN_EVAL);
    if (deep == NegaMax.this.deep) {
        NegaMax.this.bestMove = bestMove;
    }

    List<NegamaxTask> tasks = new ArrayList<>();
    for (int i = 0; i < moves.size(); i++) {
        Board copyBoard = copyAndMove(moves.get(i), board);
        GameHistory copyHistory =
        copyHistoryAndMove(copyBoard, gameHistory);
```

```

if (i == 0 || getPool().getQueuedTaskCount() > 0) {
    double eval =
        -negamax(copyBoard, copyHistory,
color.getOpposite(), deep - 1, -beta, -alpha).eval;

    if (eval > alpha) {
        alpha = eval;
        bestMove.move = moves.get(i);
        bestMove.eval = eval;
    }

    if (alpha >= beta) {
        break;
    }
} else {
    NegamaxTask task =
        new NegamaxTask(
            copyBoard,
            copyHistory,
            color.getOpposite(),
            deep - 1,
            -beta,
            -alpha,
            moves.get(i));
    task.fork();
    tasks.add(task);
}
}
for (NegamaxTask task : tasks) {
    if (task.join() == null) {
        break;
    } else {
        Node node = task.getRawResult();
        if (-node.eval > alpha) {
            alpha = -node.eval;
            bestMove.move = task.move;
            bestMove.eval = alpha;
        }
    }
}

if (alpha >= beta) {
    break;
}
}

return bestMove;
}

```

4.4 Эвристическая функция

Эвристическая функция вычисляет оценку текущего состояния доски, принимая значения от -1 до 1 в виде вещественных чисел. Отрицательная оценка указывает на преимущество оппонента, положительная — на

благоприятную ситуацию для ИИ. Эта оценка является ключевым элементом, позволяющим ИИ выбирать оптимальный ход.

Эффективность ИИ во многом зависит от структуры и параметров эвристической функции. В данном проекте используется как базовая, так и более продвинутая система оценивания состояния игровой доски.

Базовая заключается в оценивание каждой фигуры и последующим суммировании их стоимости, что позволяет получить общую оценку для каждого игрока, если найти разность, то можно понять в чью сторону перевес по фигурам. В этой системе пешка оценивается в 1 очко с увеличением стоимости по мере продвижения по доске, конь и слон оцениваются в 3 очка, ладья оценивается в 5, ферзь оценивается в 9, король оценивается в 100.

Помимо этого, применяется система бонусов и штрафов. Например, за сдвоенные пешки, которые препятствуют друг другу, начисляется штраф.

Если ИИ пропускает возможность рокировки (ходит королем или ладьей), начисляется штраф, поскольку это считается плохой практикой в шахматах.

Кроме того, реализована система бонусов за контроль центра доски. Если ИИ занимает центральные позиции, начисляется бонус к оценке; если центр занят оппонентом, начисляется штраф.

В миттельшпиле (середина игры) предусмотрены бонусы за активные фигуры и владение открытыми линиями, а также штрафы за невыведенные фигуры.

В эндшпиле (конец игры) оценка зависит от позиции короля: чем ближе король к краю доски, тем хуже для ИИ и лучше для оппонента, что позволяет учитывать безопасность короля.

Эвристическая функция, используемая в данном проекте, позволяет адаптировать поведение и оценку состояния доски в зависимости от этапа игры: начальной стадии, миттельшпиля и эндшпиля. На каждом из этих этапов ИИ применяет различные методы анализа, что позволяет ему корректировать свою стратегию на протяжении всей игры.

5 Результат

Чтобы узнать, насколько эффективен разработанный ИИ против таких же ботов сообщества, я создал аккаунт на сайте Lichess.org (рисунок 5.1) и смог по открытому API подключить свой ИИ и соответственно играть против других ботов. Так как сайт полностью открытый, то можно перейти по ссылке [6] и убедиться, что данный аккаунт действительно существует.

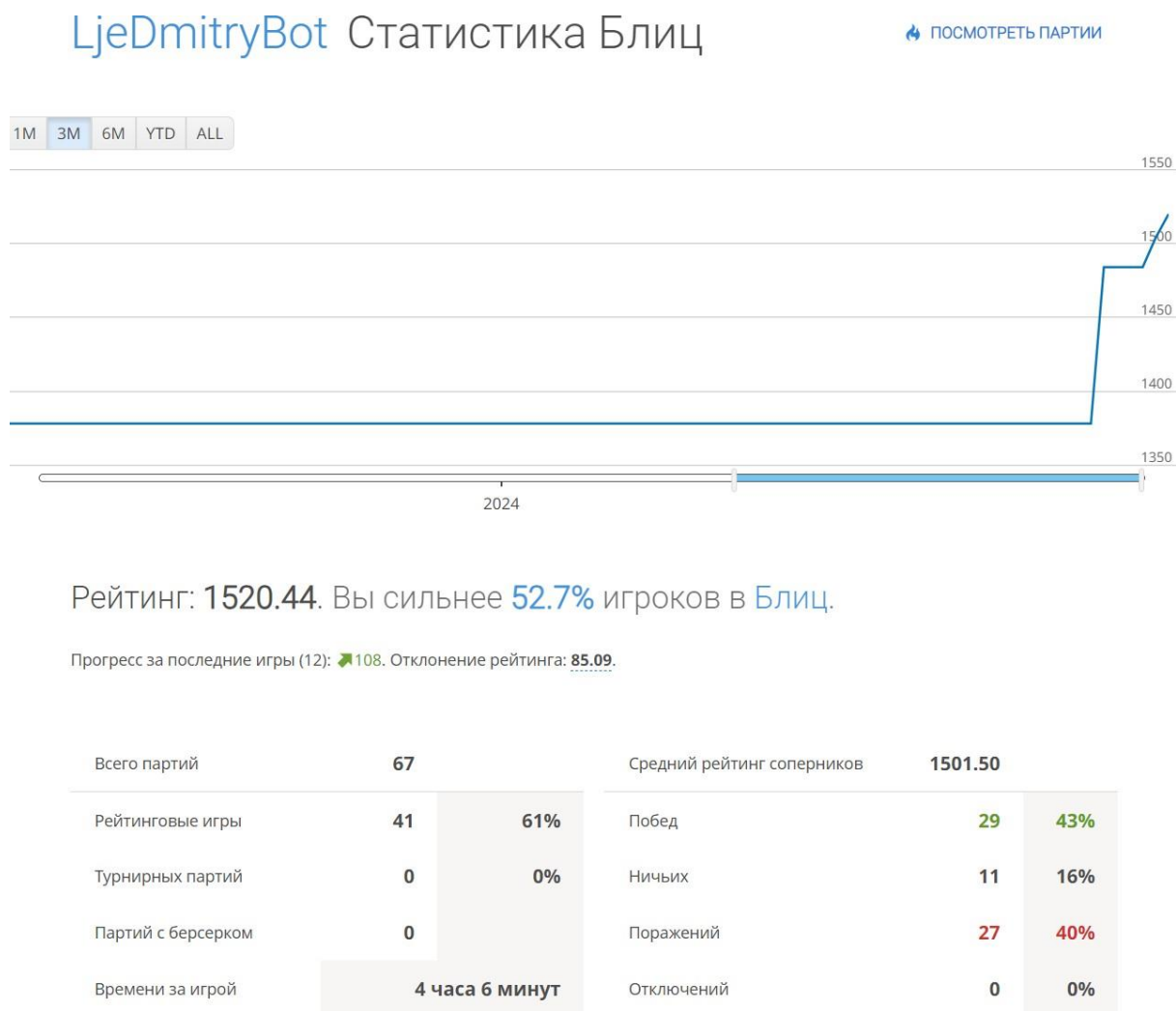


Рисунок 5.1 – Страница с профилем на сайте Lichess.org

Ниже в качестве примера я приведу повтор одной из партий, которая была сыграна против одного из ботов сообщества и в которой мой ИИ победил, играя за белых.

На рисунке 5.2 видно, что до мата черным остается два хода, так как ИИ умеет просчитывать наперед до глубины 4 (алгоритм Negamax), то соответственно он видит возможность поставить мат черным и попытается воспользоваться ей.

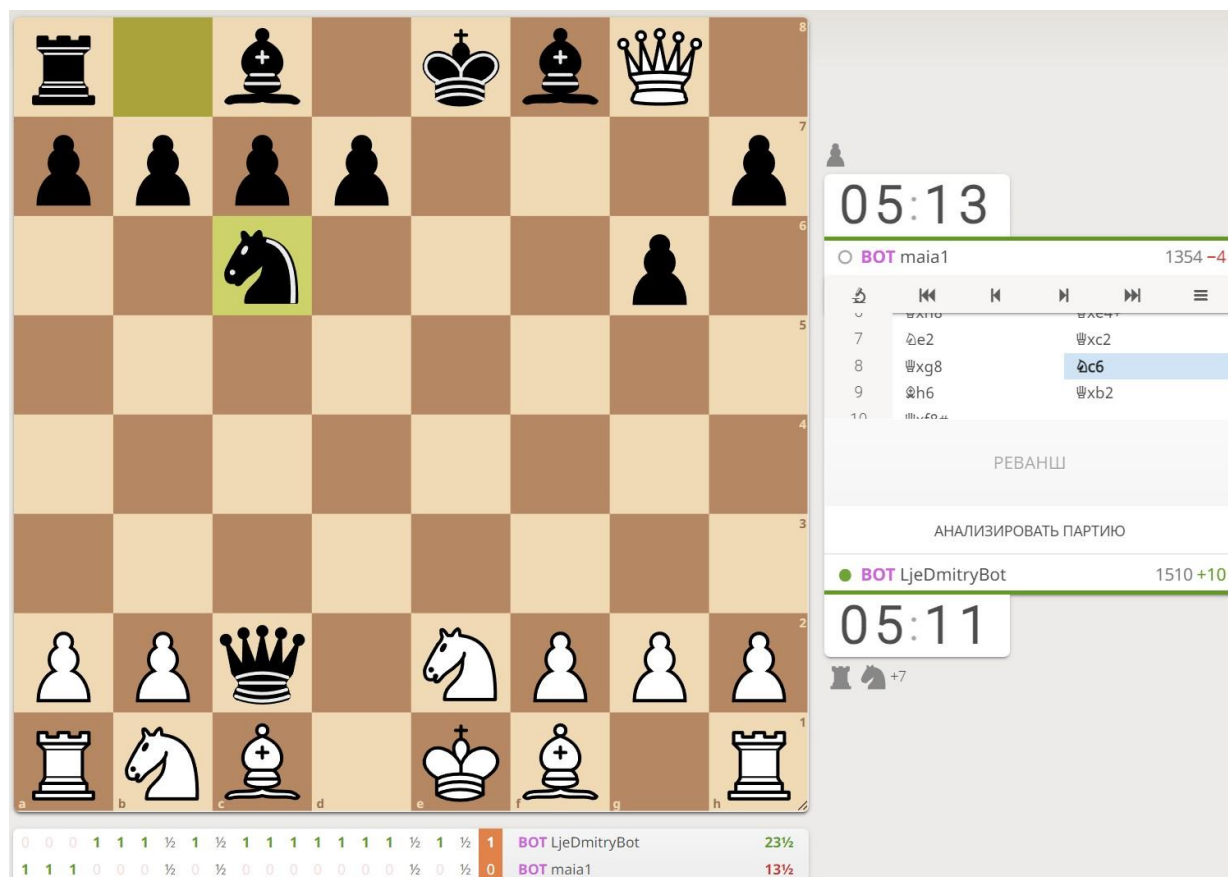


Рисунок 5.2 – Пример игры

На следующем ходе, рисунок 5.3, ИИ делает ход слоном с c1 на h6, на рисунке эти клетки подсвечиваются зеленым, тем самым обеспечивая себе мат на следующем ходу. И через ход оппонента, ИИ ставит мат черным (рисунок 5.4).

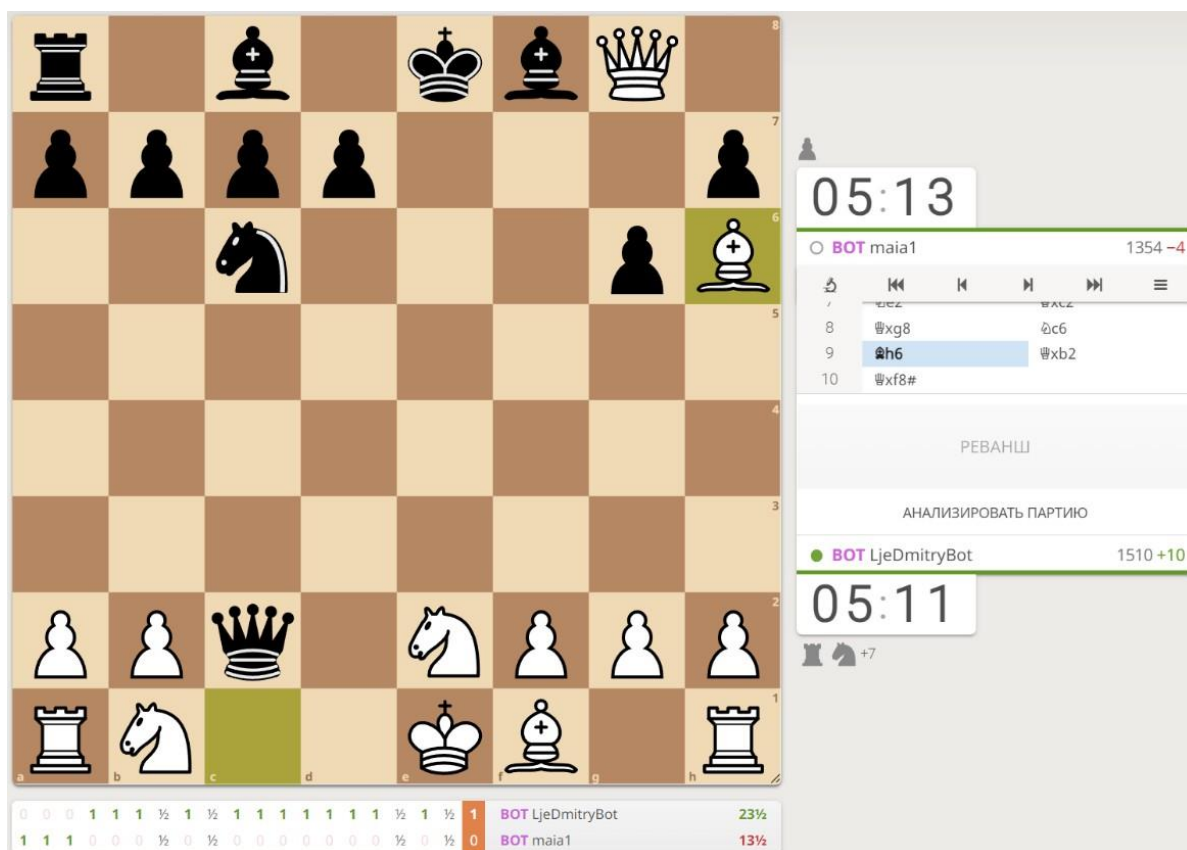


Рисунок 5.3 – Пример игры, ход слоном

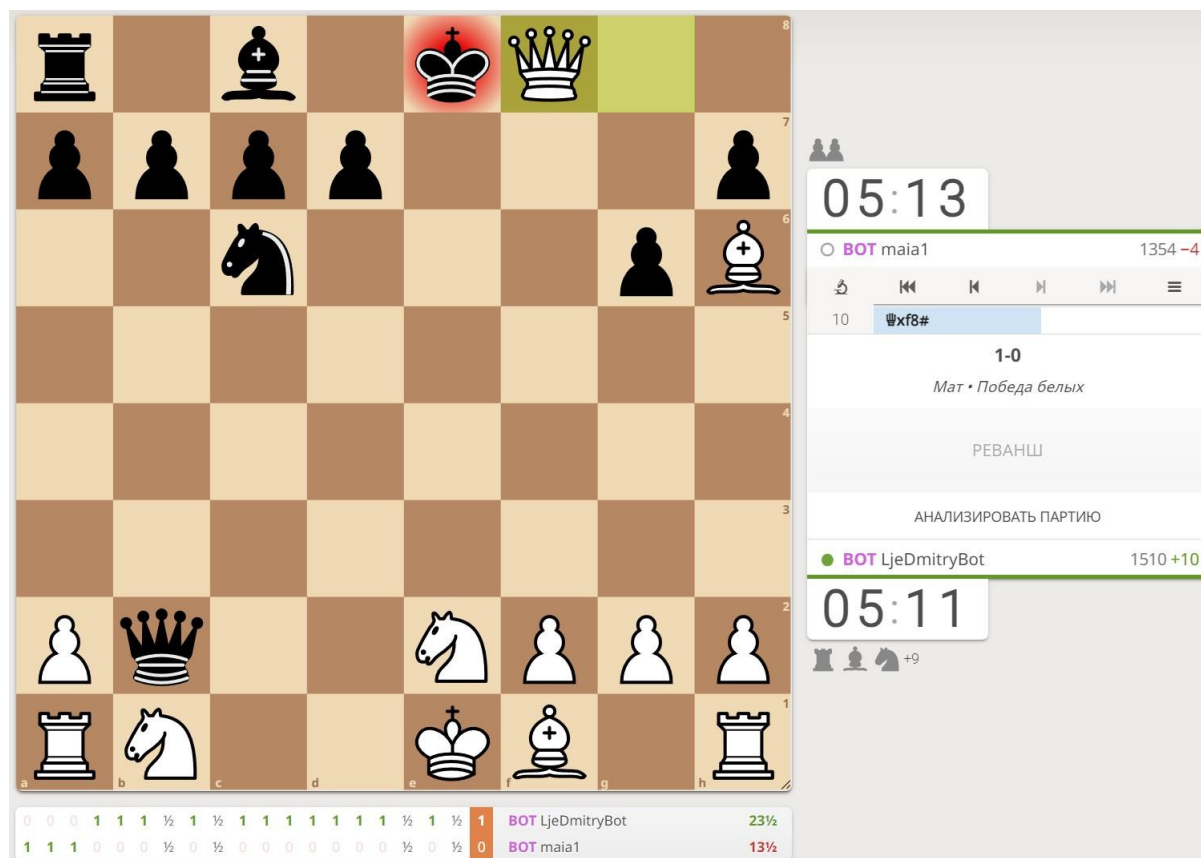


Рисунок 5.4 – Пример игры, мат черным

ЗАКЛЮЧЕНИЕ

В ходе работы был разработан шахматный движок, интегрированный с алгоритмом Minimax, Negamax и оптимизацией методом альфа-бета отсечения. Для оценки текущего состояния доски использовалась эвристическая функция, учитывающая стоимость фигур и позиционные факторы, такие как контроль центра, безопасность короля и продвижение пешек. Реализация этих компонентов позволила создать ИИ, способного принимать обоснованные и эффективные решения в различных игровых ситуациях.

После завершения разработки была проведена серия тестов, в которых ИИ играл против различных ботов на платформе Lichess.org. Результаты игр были проанализированы для определения рейтинга ИИ по сравнению с другими ботами сообщества.

Результаты анализа показали, что разработанный ИИ продемонстрировал конкурентоспособную игру, достигнув рейтинга, сопоставимого с рейтингами других ботов сообщества. Это подтверждает эффективность использованных алгоритмов и методик оценки позиций.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В пояснительной записке применяются следующие сокращения и обозначения:

ВС – вычислительные системы

СибГУТИ – Сибирский государственный университет телекоммуникаций и информатики

ИИ – искусственный интеллект

API – Application Programming Interface (интерфейс прикладного программирования)

IDE – Integrated Development Environment (интегрированная среда разработки)

DSL – Domain-Specific Language (язык программирования, который спроектирован и оптимизирован для решения задач в конкретной области или для определенного класса задач)

XML – eXtensible Markup Language (расширяемый язык разметки)

JVM – Java Virtual Machine (виртуальная машина Java)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Норвиг, П., Рассел, С. Искусственный интеллект: современный подход. – 4-е изд. – Москва: Вильямс, 2021. – 1168 с.
2. Применение Alpha/Beta в шахматных программах // ChessProgramming Wiki. 2020. URL: <https://www.chessprogramming.org/Alpha-Beta>
3. Программирование игр на Java: реализация Minimax // GeeksforGeeks. 2021. URL: <https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-1-introduction/>
4. Герберт Шилдт. Java. Полное руководство, 10-е издание = Java. The Complete Reference, 10th Edition. — М.: «Диалектика», 2018. — 1488 с.
5. Шахматные алгоритмы, которые думают почти так же, как человек, только лучше // Хабр. 2021. URL: <https://habr.com/ru/companies/skillfactory/articles/544040/>
6. lichess.org Бесплатные шахматы онлайн URL: <https://lichess.org/@/LjeDmitryBot>

ПРИЛОЖЕНИЕ А

Листинг класса Minmax

```
public class MiniMax implements Algorithm {
    private final Color botColor;
    private final int deep;
    private final boolean isMax;
    private Node bestMove;

    public MiniMax(Color color, int deep) {
        this.botColor = color;
        this.isMax = true;
        this.deep = deep;
        this.bestMove = null;
    }

    @Override
    public Move findBestMove(Board board, GameHistory
gameHistory) {
        this.bestMove = null;
        return minmax(board, gameHistory, botColor, this.deep,
MIN_EVAL, MAX_EVAL, this.isMax).move;
    }

    private Node minmax(
        Board board,
        GameHistory gameHistory,
        Color color,
        int deep,
        double alpha,
        double beta,
        boolean isMax) {
        if (deep == 0 || isGameOver(board, gameHistory)) {
            double eval = Evaluation.evaluationFunc(board,
gameHistory, botColor, isMax);
            return new Node(board.getLastMove(), eval);
        }

        List<Move> allMoves = getPossibleMoves(board, color);
        Node bestMove = new Node(allMoves.get(0), isMax ? MIN_EVAL
: MAX_EVAL);
        if (deep == this.deep) {
            this.bestMove = bestMove;
        }
        for (Move move : allMoves) {
            Board copyBoard = copyAndMove(move, board);
            GameHistory copyHistory = copyHistoryAndMove(copyBoard,
gameHistory);
            double eval =
                minmax(copyBoard, copyHistory, color.getOpposite(),
deep - 1, alpha, beta, !isMax).eval;
```

```

        if (isMax) {
            if (eval > alpha) {
                alpha = eval;
                bestMove.move = move;
                bestMove.eval = eval;
            }
        } else {
            if (eval < beta) {
                beta = eval;
                bestMove = new Node(move, eval);
            }
        }

        if (beta <= alpha) {
            break;
        }
    }

    return bestMove;
}

@Override
public Move getBestMoveAfterTimeout() {
    return bestMove.move;
}
}

```

ПРИЛОЖЕНИЕ Б

Листинг класса NegaMax

```
public class NegaMax implements ParallelAlgorithm {
    private final Color botColor;
    private final int deep;
    private final ForkJoinPool pool;

    private Node bestMove;

    public NegaMax(Color color, int deep) {
        this.botColor = color;
        this.deep = deep;
        this.bestMove = null;
        this.pool = new
ForkJoinPool(Runtime.getRuntime().availableProcessors() / 2 +
1);
    }

    public NegaMax(Color color, int deep, int parallelism) {
        this.botColor = color;
        this.deep = deep;
        this.bestMove = null;
        this.pool = new ForkJoinPool(parallelism);
    }

    @Override
    public Move findBestMove(Board board, GameHistory
gameHistory) {
        bestMove = null;
        NegaMaxTask task =
            new NegaMaxTask(board, gameHistory, botColor, deep,
MIN_EVAL, MAX_EVAL, null);

        return pool.invoke(task).move;
    }

    private class NegaMaxTask extends RecursiveTask<Node> {
        private final Board board;
        private final GameHistory gameHistory;
        private final Color color;
        private final int deep;
        private final double beta;
        private final double alpha;
        private final Move move;
    public NegaMaxTask(
        Board board,
        GameHistory gameHistory,
        Color color,
        int deep,
        double alpha,
        double beta,
        Move move) {
```

```

        this.board = board;
        this.gameHistory = gameHistory;
        this.color = color;
        this.deep = deep;
        this.alpha = alpha;
        this.beta = beta;
        this.move = move;
    }
    private Node negamax(
        Board board, GameHistory gameHistory, Color color, int
        deep, double alpha, double beta) {
        if (deep == 0 || isGameOver(board, gameHistory)) {
            double signEval = color == botColor ? 1.0 : -1.0;
            boolean isBotMove = color == botColor;

            return new Node(
                board.getLastMove(),
                Evaluation.evaluationFunc(board, gameHistory,
                botColor, isBotMove) * signEval);
        }

        List<Move> moves = getPossibleMoves(board, color);
        Node bestMove = new Node(moves.get(0), MIN_EVAL);
        if (deep == NegaMax.this.deep) {
            NegaMax.this.bestMove = bestMove;
        }

        List<NegamaxTask> tasks = new ArrayList<>();
        for (int i = 0; i < moves.size(); i++) {
            Board copyBoard = copyAndMove(moves.get(i), board);
            GameHistory copyHistory = copyHistoryAndMove(copyBoard,
            gameHistory);

            if (i == 0 || getPool().getQueuedTaskCount() > 0) {
                double eval =
                    -negamax(copyBoard, copyHistory,
                    color.getOpposite(), deep - 1, -beta, -alpha).eval;

                if (eval > alpha) {
                    alpha = eval;
                    bestMove.move = moves.get(i);
                    bestMove.eval = eval;
                }

                if (alpha >= beta) {
                    break;
                }
            }
        }
    }

```

```

        } else {
            NegamaxTask task =
                new NegamaxTask(
                    copyBoard,
                    copyHistory,
                    color.getOpposite(),
                    deep - 1,
                    -beta,
                    -alpha,
                    moves.get(i));
            task.fork();
            tasks.add(task);
        }
    }

    for (NegamaxTask task : tasks) {
        if (task.join() == null) {
            break;
        } else {
            Node node = task.getRawResult();
            if (-node.eval > alpha) {
                alpha = -node.eval;
                bestMove.move = task.move;
                bestMove.eval = alpha;
            }
        }

        if (alpha >= beta) {
            break;
        }
    }

    return bestMove;
}

@Override
protected Node compute() {
    if (alpha >= beta) {
        return null;
    }
    return negamax(board, gameHistory, color, deep, alpha,
beta);
}

@Override
public void shutdownPool() {
    pool.shutdown();
}

public boolean isShutdownPool() {
    return pool.isShutdown();
}

@Override
public Move getBestMoveAfterTimeout() {
    return bestMove.move;
}
}

```

ПРИЛОЖЕНИЕ В

Листинг класс Evaluation

```
class Evaluation {
    public static final double MAX_EVAL = 1.0;

    public static final double MIN_EVAL = -1.0;

    private static final Map<FigureType, Double> PIECE_PRICE =
        Map.of(
            FigureType.PAWN,
            1.0,
            FigureType.KNIGHT,
            3.0,
            FigureType.BISHOP,
            3.0,
            FigureType.ROOK,
            5.0,
            FigureType.QUEEN,
            9.0,
            FigureType.KING,
            100.0);

    public static double evaluationFunc(
        Board board, GameHistory gameHistory, Color botColor,
        boolean botMove) {
        if (GameStateChecker.isMate(board,
            botColor.getOpposite())) {
            return MAX_EVAL;
        } else if (GameStateChecker.isMate(board, botColor)) {
            return MIN_EVAL;
        }

        if (GameStateChecker.isDraw(board, gameHistory)) {
            return 0;
        }

        double ourRate = evaluationBoard(board, gameHistory,
            botColor);
        double enemyRate = evaluationBoard(board, gameHistory,
            botColor.getOpposite());
        if (!botMove) {
            ourRate += pieceExchange(board, botColor);
        } else {
            enemyRate += pieceExchange(board,
                botColor.getOpposite());
        }
        double result = (ourRate - enemyRate) * (1 + 10 / (ourRate
            + enemyRate)) / 1000;
        result = Math.round(result * 1e9) / 1e9;
        return result;
    }
}
```

```

    protected static double evaluationBoard(Board board,
GameHistory gameHistory, Color color) {
        return kingEndgameEval(board, color)
            + calculatePiecesPrice(board, color)
            + Bonuses.middlegame(board, gameHistory, color)
            + Bonuses.openLines(board, color,
gameHistory.getMoves().size());
    }

    protected static double kingEndgameEval(Board board, Color
color) {
        Position kingPos = board.getKingPositionByColor(color);
        Position enemyKingPos =
board.getKingPositionByColor(color.getOpposite());
        if (board.getAllPiecePositionByColor(color).size() != 1) {
            return 0.0;
        }

        int row = kingPos.row().value();
        int col = kingPos.col().value();

        int rowCenter = row <= 3 ? 3 : 4;
        int colCenter = col <= 3 ? 3 : 4;
        int rowDistForCenter = Math.abs(row - rowCenter);
        int colDistForCenter = Math.abs(col - colCenter);

        int rowDistForEnemyKing = Math.abs(row -
enemyKingPos.row().value());
        int colDistForEnemyKing = Math.abs(col -
enemyKingPos.col().value());

        int moveCount = board.getPiece(kingPos).getAllMoves(board,
kingPos).size();

        return (rowDistForCenter + colDistForCenter) * -1
            + ((moveCount + rowDistForEnemyKing +
colDistForEnemyKing) * 0.1);
    }

    protected static double calculatePiecesPrice(Board board,
Color color) {
        return board.getAllPiecePositionByColor(color).stream()
            .map(pos -> calculatePiecePrice(board, pos, color))
            .reduce(0.0, Double::sum);
    }

    protected static double calculatePiecePrice(Board board,
Position pos, Color color) {
        return board.getPiece(pos).getFigureType() ==
FigureType.PAWN
            ? calculatePawnPrice(board, pos, color)
            : PIECE_PRICE.get(board.getPiece(pos).getFigureType());
    }

```

```

private static double calculatePawnPrice(Board board,
Position pos, Color color) {
    int row = pos.row().value();
    int col = pos.col().value();

    double rowPrice =
        PIECE_PRICE.get(FigureType.PAWN) + (color ==
Color.WHITE ? row : 7 - row) * 0.1 - 0.1;
    double centerBonus =
        col > 2 && col < 5 && row > 2 && row < 5 ? (4 - (col
% 4 + col % 3)) * 0.1 : 0;
    double doublePawnPenalty = 0;
    if (board.hasPiece(col, row + 1)
        && board.getPiece(col, row + 1).getFigureType() ==
FigureType.PAWN) {
        doublePawnPenalty += 0.5;
    }
    if (board.hasPiece(col, row - 1)
        && board.getPiece(col, row - 1).getFigureType() ==
FigureType.PAWN) {
        doublePawnPenalty += 0.5;
    }

    return rowPrice + centerBonus - doublePawnPenalty;
}

protected static double pieceExchange(Board board, Color
color) {
    double result = 0;
    boolean isSecurity;
    List<Position> friendlies =
board.getAllPiecePositionByColor(color);
    List<Position> enemies =
board.getAllPiecePositionByColor(color.getOpposite());

    for (Position friendly : friendlies) {
        if
(!friendly.equals(board.getKingPositionByColor(color))) {
            isSecurity = isSecurity(board, friendly, color);
        }
    }
}

```

```

        for (Position enemy : enemies) {
            Piece enemyPiece = board.getPiece(enemy);
            FigureType promotionPiece = null;
            if (enemyPiece.getFigureType() == FigureType.PAWN
                && (friendly.row().value() == 0 ||
friendly.row().value() == 7)) {
                promotionPiece = FigureType.QUEEN;
            }

            Move move = new Move(enemy, friendly, promotionPiece);
            if (!isSecurity && enemyPiece.canMove(board, move)) {
                result -= calculatePiecePrice(board, friendly,
color);
                break;
            }
        }
    }
    return result;
}

protected static boolean isSecurity(Board board, Position pos,
Color color) {
    List<Position> friendlies =
board.getAllPiecePositionByColor(color);

    for (Position friendly : friendlies) {
        if (!friendly.equals(pos)) {
            Piece piece = board.getPiece(friendly);
            FigureType promotionPiece = null;
            if (piece.getFigureType() == FigureType.PAWN
                && (pos.row().value() == 0 || pos.row().value() ==
7)) {
                promotionPiece = FigureType.QUEEN;
            }

            Move move = new Move(friendly, pos, promotionPiece);
            if (piece.canMove(board, move, true, false)) {
                return true;
            }
        }
    }
    return false;
}
}

```